CHAPTER 37

Missing Categorical Data

Most Frequent, Constant, Missing Category Strategies

CampusX — 100 Days of Machine Learning
Feature Engineering Series, Videos 31–40

37.1 Missing Categorical Values

Categorical features can also contain missing values. Examples include a "city" column where some rows are blank, a "gender" column with unrecorded responses, or a "product_category" column where the category was not assigned. The strategies for handling missing categorical data differ from numerical imputation because we cannot compute a "mean" of text labels.

37.2 Strategies for Missing Categorical Data



Figure 37.1: Four strategies for handling missing categorical data. Each produces a different distribution of categories after imputation.

Strategy	What it does	When to use	Risk
Most Frequent	Replace NaN with the modal category	Missingness is low and one category dominates	Inflates the dominant category
Constant	Replace NaN with a fixed string (e.g., "Unknown")	Missingness might be informative	Adds an artificial category
Missing Category	Create a new "(Missing)" category	The fact that a value is missing may predict the target	High cardinality if applied to many features
Drop	Remove rows with missing categories	Very low missingness and MCAR	Data loss (same as CCA)

Table 37.1: Strategies for missing categorical data.

37.3 Implementation

37.3.1 Most Frequent Imputation

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

# Categorical data with missing values
df = pd.DataFrame({
'city': ['NYC', 'LA', 'NYC', np.nan, 'Chicago', np.nan,
'NYC', 'Houston', np.nan, 'LA', 'NYC', 'Chicago'],
'gender': ['M', 'F', np.nan, 'F', 'M', 'M',
np.nan, 'F', 'M', np.nan, 'F', 'M'],
'subscription': ['Basic', 'Premium', 'Basic', np.nan, 'Premium',
'Basic', 'Premium', np.nan, 'Basic', 'Premium', 'Basic', 'Premium']
})

print("=== MISSING VALUES ===")
print(df.isnull().sum())
print()

# Most frequent imputation
imputer = SimpleImputer(strategy='most_frequent')
df_imputed = pd.DataFrame(
imputer.fit_transform(df),
columns=df.columns
)

print("Most frequent values (statistics_):", imputer.statistics_)
print("\n=== AFTER MOST FREQUENT IMPUTATION ===")
print(df_imputed)
print("\nCategory counts after imputation:")
for col in df.columns:
print(f"\n{col}:")
print(df_imputed[col].value_counts())
```

Listing 37.1: Most frequent imputation for categorical data.

37.3.2 Constant / Missing Category Imputation

```
# Constant imputation — replace with a designated string
imputer_const = SimpleImputer(strategy='constant', fill_value='Unknown')
df_const = pd.DataFrame(
imputer_const.fit_transform(df),
columns=df.columns
)
print("=== CONSTANT IMPUTATION (fill_value='Unknown') ===")
print(df_const)
print()

# Manual approach: create a missing category
df_missing_cat = df.copy()
for col in df.columns:
df_missing_cat[col] = df_missing_cat[col].fillna('(Missing)')

print("=== MISSING CATEGORY APPROACH ===")
print(df_missing_cat)
print("\nCategory counts with missing category:")
for col in df.columns:
```

```
print(f"\n{col}:")
print(df_missing_cat[col].value_counts())
```

Listing 37.2: Constant and missing-category imputation.

37.4 Consequences of Different Approaches

37.4.1 Most Frequent Imputation

- **Pro:** Simple, preserves the existing category set.
- **Con:** Inflates the frequency of the most common category. If "NYC" was already 40% of the data and you impute 20 more NaNs as "NYC", it might become 55% — distorting the true distribution.
- **Con:** Breaks the relationship between the feature and other features. If missingness depends on another feature (MAR), ignoring that dependence introduces bias.

37.4.2 Constant / Missing Category

- **Pro:** Preserves information that a value was missing (the missingness itself might be predictive).
- **Pro:** Does not inflate any existing category.
- **Con:** Adds a new category, increasing cardinality. For high-cardinality features, this can be problematic for one-hot encoding.
- **Con:** Tree-based models can handle the new category natively, but linear models require encoding — and the encoding of "(Missing)" carries no inherent ordinal meaning.

> **Common Mistake**
> Do not use `strategy="mean"` or `"median"` on categorical (string) columns. These strategies are designed for numerical data. Scikit-learn will either raise an error or produce meaningless results. Always use `"most_frequent"` or `"constant"` for categorical features.

37.5 Key Takeaways

> **★ For missing categorical data, use most_frequent imputation (SimpleImputer strategy="most_frequent") when the dominant category is appropriate, or create a dedicated "(Missing)" category when the missingness itself may be informative. Never apply mean/median strategies to categorical columns.**